

Option #2: Closed-Domain Customer-Service NLP Chatbot Final Version

Justan Dunn

Colorado State University Global

CSC525: Principles of Machine Learning

Dong Nguyen

July 2, 2026

Option #2: Closed-Domain Customer-Service NLP Chatbot Final Version

This portfolio project presents the final version of a closed-domain customer-service routing and retrieval chatbot. The system accepts a natural-language support message, estimates the most likely service route, and returns a controlled response associated with that route. When evidence is weak, it does not invent an answer; it asks the user to rephrase the request or sends the case toward human support. The project applies natural language processing through character n-gram term-frequency inverse-document-frequency (TF-IDF) features, supervised class centroids, cosine similarity, and calibrated fallback thresholds. Its purpose is to demonstrate an explainable chatbot that responds meaningfully while making uncertainty visible.

The final design also corrects an important limitation discovered in the earlier milestones. The Module 5 evaluation produced perfect accuracy when curated route keywords were appended to held-out questions, but normal users do not provide those keyword banks. The final model therefore trains on customer-language questions and is evaluated against separate raw-message calibration and test sets. This change produces lower but more credible results and supports the central conclusion of the project: a controlled NLP chatbot can route common support requests effectively, but it still requires fallback logic, broader data, and human escalation before enterprise use.

Project Goal and Domain

The chatbot's defined goal is to identify the correct support function for a customer message and provide an approved response that is clearly related to the request. It supports ten routes: account access, billing and invoices, human escalation, onboarding, order status, policy lookup, returns and refunds, service scheduling, technical support, and training materials. These routes represent common operational needs that can be adapted to different companies without exposing actual customer or company data.

The chatbot is closed-domain rather than open-domain. This means that it is designed

to answer only within the approved support categories and does not attempt general conversation, current-events questions, or unrestricted text generation. A closed-domain design is appropriate because customer-service answers frequently depend on controlled policies and standard operating procedures. The system retrieves a route-specific response instead of composing a new policy. That constraint reduces nonsense responses and creates a clearer audit trail from an input decision to an approved support action.

Dataset Development and Milestone Continuity

The project developed through four milestones. Module 3 established an enterprise knowledge-support chatbot using TF-IDF and similarity ranking. Module 4 refined the use case into customer-service routing and selected a hybrid data strategy: the RSiCS corpus for relational customer language, WikiQA for answer relevance and no-answer behavior, and a sanitized FAQ/SOP corpus for approved domain responses. Beaver et al. (2017) demonstrate that greetings, backstory, emotion, and other relational language can obscure a customer's underlying intent. Yang et al. (2015) similarly show the importance of distinguishing relevant candidate answers from examples for which no candidate answer is correct. Those findings shaped the final use of neutral conversational augmentation and explicit out-of-domain tests.

Module 5 created 80 sanitized FAQ/SOP questions across the ten routes. Module 6 then produced a command-line alpha that returned approved answers and used a similarity threshold for fallback. The final implementation retains that controlled corpus but separates data roles more rigorously. The 80 original questions are the only labeled content used for route learning. Each question is also wrapped in two route-neutral conversational frames, producing 240 training examples without inserting label names or route-specific keywords. A separate 52-message calibration set selects the feature configuration and thresholds. A final 62-message test set contains 50 in-domain messages and 12 out-of-domain messages. No exact training question appears in either evaluation set. RSiCS and WikiQA inform the evaluation design but are not represented as direct training rows in the final local model.

NLP Model, Tools, and Libraries

The final chatbot is implemented in Python 3 with standard-library modules including csv, json, re, math, argparse, pathlib, and collections. No external runtime library is required. The trained model is stored as transparent JSON rather than as an opaque binary object, allowing the vocabulary, inverse document-frequency values, route centroids, responses, and decision thresholds to be inspected directly. Supporting scripts reproduce dataset construction, model selection, evaluation, and validation.

TF-IDF represents text by increasing the weight of terms that are informative within the collection while reducing the influence of terms that appear in many documents. Salton and Buckley (1988) established term weighting as a strong baseline for automatic text retrieval, and Manning et al. (2008) explain how TF-IDF vectors and cosine similarity support ranked retrieval and vector-space classification. The final implementation applies these ideas to character n-grams of length three through five. Character n-grams capture word fragments and are more tolerant of inflection, minor spelling variation, and differences between words such as invoice, invoicing, and invoices than exact unigram matching alone.

During training, every example is converted to a normalized TF-IDF vector. Vectors sharing a route label are averaged and normalized to create a supervised route centroid. At runtime, a raw customer message is transformed with the same feature configuration and compared with every centroid using cosine similarity. The highest-scoring route is the proposed intent, and the difference between the highest and second-highest score is the decision margin. The chatbot answers only when both the highest score and the margin exceed calibrated thresholds.

Hyperparameter Selection and Fallback Control

The training process compared four feature configurations: word unigrams, word unigrams and bigrams, character 3-5-grams, and a hybrid word-character representation. For each configuration, the program evaluated 20 minimum-score values and nine minimum-margin values, producing 720 calibration combinations. To prevent a model from obtaining a high score by rejecting nearly every message, eligible configurations were required to preserve at least 70% in-domain coverage and reject at least 60% of out-of-domain calibration inputs. The selected configuration was character 3-5-gram TF-IDF, with 4,706 features, a minimum similarity of 0.160, and a minimum margin of 0.020.

On the calibration set, the selected model achieved an overall decision accuracy of 0.8077, an in-domain coverage of 0.8500, and an out-of-domain rejection rate of 0.8333. Confidence-based rejection follows the practical principle that uncertain or out-of-distribution inputs should be detectable rather than automatically forced into a known class. Hendrycks and Gimpel (2017) describe confidence as a useful baseline signal for identifying misclassified and out-of-distribution examples. In this project, the score and margin are not treated as probabilities; they are decision signals used to choose between a controlled answer and safe fallback.

Final Test Results

Table 1 summarizes the final results. Raw route accuracy measures whether the highest-scoring centroid matches the expected route before fallback. In-domain coverage measures how often the chatbot answers instead of rejecting a support message. Accepted route accuracy measures the correctness of answered in-domain messages. Out-of-domain rejection measures how often unrelated questions are sent to fallback. Overall decision accuracy counts a correct accepted route or a correct out-of-domain rejection as a successful decision.

Table 1
Final Raw-Message Test Results

Metric	Result
Raw route accuracy	0.9200
In-domain coverage	0.9000
Accepted route accuracy	0.9333
Out-of-domain rejection rate	0.7500
Overall decision accuracy	0.8226

Note. The final test contains 50 in-domain and 12 out-of-domain raw messages. No route labels or curated keyword banks were appended to inputs.

The final model correctly ranked the route for 92% of in-domain messages. After fallback control, it answered 45 of 50 in-domain messages and routed 42 of those accepted messages correctly. It rejected 9 of 12 unrelated messages. The remaining errors included 5 false rejections, 3 false accepts, and three accepted misroutes. These errors are retained in the row-level prediction file rather than hidden from the submission.

The results improve substantially on the fair question-only baseline reported in Module 5 while avoiding the unsupported claim of perfect performance. The remaining false accepts show that lexical similarity alone cannot guarantee domain awareness. For example, a general request that contains words such as time, solve, or explain can resemble policy or training language even when the topic is unrelated. A production system would need a larger set of real, sanitized customer messages, dedicated outlier examples, probability calibration, monitoring, and periodic human review.

Chatbot Behavior and Running Instructions

The final program provides interactive, single-message, demonstration, and JSON modes. In interactive mode, the user enters a message, the model predicts a route, and the program prints the approved response, human-readable route, confidence label, and similarity score. The commands /topics, /help, and /quit support navigation. The prepared demonstration contains seven representative in-domain requests and one unrelated request that activates fallback.

The instructor can open the extracted project folder and double-click run_chatbot.bat on Windows. The same interface can be started from PowerShell or another terminal with the following command:

```
python chatbot.py --interactive
```

The prepared demonstration runs with the command below and writes a transcript to the outputs folder:

```
python chatbot.py --demo
```

No installation step is required beyond Python 3.10 or newer. The included README provides additional commands for one-message and JSON output, and the validation script confirms model, data, report, transcript, and metric files.

Limitations and Responsible Use

The project is a coursework prototype built from synthetic, sanitized examples. It does not use private customer records, production policies, or current company systems. Its approved responses are generic workflow guidance rather than actual service commitments. A company adapting this design would need policy-owner approval, privacy controls, access management, secure logs, accessibility review, and testing across the language used by its customers.

The classifier also predicts only one primary route. Multi-intent messages, long conversations, contextual follow-up questions, multilingual inputs, and changing policies

remain outside the final scope. These limits are appropriate for a small explainable project, but they prevent the model from being treated as an autonomous customer-service replacement. The fallback response and human escalation route are therefore core design features rather than exceptions.

Conclusion

The final chatbot satisfies Option #2 by providing a runnable NLP system that responds to customer-service inputs with relevant, controlled content. It is closed-domain, dependency-free, reproducible, and documented with complete run instructions. More importantly, the final evaluation uses raw messages and reports residual errors instead of relying on metadata-assisted accuracy. The character n-gram TF-IDF centroid model achieved strong routing performance for the defined support routes while the calibrated thresholds rejected most unrelated inputs. The project demonstrates a practical foundation that other companies could adapt, provided that deployment adds organization-specific data, governance, security, monitoring, and human oversight.

Use of AI Assistance

AI assistance was used to help structure code, organize project artifacts, and draft portions of the written report. The scripts were executed locally, the reported metrics were generated from the included data, and the final package was validated in the CSC525 workspace. The student remains responsible for reviewing the submitted work and explaining its methods and limitations.

References

- Beaver, I., Freeman, C., & Mueen, A. (2017). *An annotated corpus of relational strategies in customer service*. arXiv. <https://doi.org/10.48550/arXiv.1708.05449>
- Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations*. <https://arxiv.org/abs/1610.02136>
- Manning, C. D., Raghavan, P., & Schutze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- Yang, Y., Yih, W.-t., & Meek, C. (2015). WikiQA: A challenge dataset for open-domain question answering. In L. Marquez, C. Callison-Burch, & J. Su (Eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing* (pp. 2013-2018). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D15-1237>